Encryption is a Two-Way Street: The RSA Algorithm

Alberta Williams

Department of Mathematics, Southern New Hampshire University

MAT 260: Cryptology

Dr. Brian Stout

June 21, 2026

Encryption is a Two-Way Street: The RSA Algorithm

With the development of computer networks came the necessity for cryptosystems that would provide security without using the impractical approach of exchanging symmetric keys. To solve the problem of how to exchange keys securely without any interceptions from third parties, Diffie and Hellman invented the concept of public-key cryptography. This was an innovative way to keep your private key secret and at the same time share your public key with everyone who wants to communicate with you in a secure way. Taking advantage of this innovative approach, Ron Rivest, Adi Shamir, and Leonard Adleman invented the RSA cryptosystem. This paper will examine the RSA cryptosystem and consider the following topics: the algorithm, the number theory behind the trapdoor function and the security of RSA.

When the use of computer networks arose, people needed to communicate securely using an agreed upon encryption system. The problem was how the parties could share the keys while keeping it secret so their messages weren't discovered by a third-party observer. This is symmetric key cryptography and it is impractical to share the same key in a computer system. In 1976, Diffie and Hellman proposed a solution to this problem. The two parties still share the encryption algorithm, but now they will maintain a private key and a public key. In a public-key cryptosystem, a private key is only known by the owner, and a public key can be accessed by anyone who wants to communicate with the owner. This eliminates the need for the parties to share a secret key in advance (Luo et al., 2023).

Ron Rivest, Adi Shamir, and Leonard Adleman developed the RSA algorithm based on the Diffie-Hellman key exchange system. Conceptually, public key cryptography can be thought of as using a locked mailbox If one party, Alice, wants to send another party, Bob, a secret message, she can put it in Bob's mailbox slot. The slot is the public key. It is openly shared for the purpose

of allowing others to send Bob information securely. Putting the letter into the slot is like encryption. To read his mail, Bob uses a physical key to open the mailbox. This is the private key that only he knows. Reading the mail is like decryption. The strength of the RSA algorithm lies in the difficulty of prime factorization. Prime factorization means decomposing an integer into a product of primes. Part of the process of generating the public key in RSA is multiplying two large primes to get a modulus. A high strength RSA key requires 4096 bits. There is no known efficient algorithm that can find the factors. The world's most powerful computers would need millions of years to find the two factors (Luo et al., 2023).

The steps to encrypt and decrypt a message using the RSA algorithm will be described next. First, two large random prime numbers *p* and *q* are chosen and multiplied together to get a modulus *n*. Next, a large number *e* is chosen that is relatively prime to *(p - 1)(q - 1)*. The numbers *n* and *e* make up the public key. This is analogous to Bob and his mailbox in the aforementioned example. The number *d* is generated such that ed = 1 (mod (p -1)(q - 1)). The number *d* and *n* constitute the private key. The sender of the message first converts the message to a sequence of numbers. Then encrypts the message *M* using the receiver's public key applying the formula $M^e (mod\ n)$ and sends the ciphertext *C* to the receiver. The receiver decrypts the message using their private key with the formula $C^d (mod\ n)$ (Luo et al., 2023).

The RSA algorithm relies on number theory to create a "trapdoor" function—a calculation that is easy to perform in one direction but extremely difficult to reverse. Euler's Totient function counts how many positive integers up to n are relatively prime (coprime) to *n*. Two numbers are relatively prime if their greatest common divisor equals one. The Totient's function primary purpose in RSA is to bridge the gap between modular exponentiation and the identity of a number. When you have two prime numbers *p* and *q*, the modulus is $n = p \times q$. The Totient

function of this product is $(p - 1)(q - 1)$ or $phi(n)$. Because factoring large numbers is computationally infeasible, if an attacker only knows the public number n, they cannot easily calculate the Totient function. Knowing the Totient function is required to calculate the private key (Tech With Nikola, 2023).

The mathematics behind decryption stems from Euler's Theorem, which states that if $gcd(a, n) = 1$, $a^{phi(n)} = 1 \pmod n$. In RSA we want to get back to our original number x. To do this we use the property: $x^{k*phi(n) + 1} = x \pmod n$. By choosing an encryption key e and a decryption key d such that $ed = \pmod{phi\ (n)}$ (Tech With Nikola, 2023). We will look at an example of encrypting and decrypting a message. Alice wants to send the message "hi" to Bob. We will use ASCII to convert the message "hi" to the number 104105. Bob chooses p to be 1721 and q to be 1801. The modulus n = 3099521. The $phi(n)$ is 1720 x 1800 = 3096000. Bob chooses e to be 1012333. Based on the equation $ed = 1 \pmod{phi\ (n))}$ d is 997. Alice sends the ciphertext to Bob which is $104105^{1012333}$ (mod 3099521) equaling 1172046. Bob will decrypt the message by calculating 1172046^{997}(mod 3099521) which gives us 104105.

One form of attack on RSA is a probabilistic algorithm that seeks to factorize n. In such an attack, once one determines the original primes (p and q) used to generate n, he or she will use the same to derive the private key needed. After the derivation of the private key, the encryption will be broken. However, in this algorithm, the attacker does not guess the original primes p and q randomly. They seek to find an exponent m such that $a^m = 1 \pmod n$ and $a^{m/2} \neq 1 \pmod n$. Should the attacker succeed in finding an exponent m satisfying the two conditions, they will use Euclid's algorithm to determine the $gcd\ (a^{m/2 - 1}, n)$. This calculation is highly likely to yield a non-trivial factor of n (Dewi et al., 2024).

Although mathematically accurate, it is evident that such an attack cannot realistically occur on real-world RSA cryptography. The estimated time complexity for the algorithm was found to be O(n log n), which is theoretically relatively quick. However, the results of the authors' empirical research proved the opposite. As the more the keys increased in size, the faster the process became increasingly slow. Specifically, the attack was tried on the authors' laptop. They succeeded in cracking a tiny 16-bit key, which however consumed on average over two minutes. In case of cracking an 18-bit key, the laptop was unable to cope with such computation as it lacked processing power; as a result, the system crashed because of "MemoryError." For comparison, whereas the authors were unable to crack even a simple 18-bit key, RSA actually utilizes 1024 and 2048 bit keys (Dewi et al., 2024).

In the end, what makes RSA strong is the complexity of factoring large integers. As shown above, although there exists some probabilistic algorithms that try to find the original private key, the sheer amount of processing power and memory necessary to handle current large key sizes renders these methods infeasible. Nevertheless, the very same mathematical basis behind the RSA encryption's security becomes the target for its ultimate demise. For the future of cryptography holds an immense paradigm shift – the rise of large-scale quantum computing. In contrast to the sub-exponential complexity in factoring large integers by classical computers, quantum computers running algorithms such as Shor's Algorithm will be able to crack a 2048-bit RSA key in a matter of hours or even minutes (Cain et al., 2026).

References

Cain, M., Xu, Q., King, R., Picard, L. R. B., Levine, H., Endres, M., Preskill, J., Huang, H.-Y., & Bluvstein, D. (2026). *Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits*. https://doi.org/10.48550/arXiv.2603.28627

Dewi, S. T., Yolandito, R. V., & Faisal. (2024). Time Complexity Analysis of Attacking RSA Using Probabilistic Factorization Algorithm in Python. *2024 8th International Conference on Information Technology, Information Systems and Electrical Engineering (ICITISEE), Information Technology, Information Systems and Electrical Engineering (ICITISEE), 2024 8th International Conference On*, 208–213. https://doi.org/10.1109/ICITISEE63424.2024.10730375

Luo, Z. J., Liu, R., Mehta, A., & Ali, M. L. (2023). Demystifying the RSA Algorithm: An Intuitive Introduction for Novices in Cybersecurity. *The Journal of Computing Sciences in Colleges, 2024 The Journal of Computing Sciences in Colleges*. https://doi.org/10.48550/arXiv.2308.02785

Tech With Nikola. (2023, August 13). *RSA Algorithm Explained + implementation* [Video]. YouTube. https://www.youtube.com/watch?v=nvcssTsiavg